

GRADUATION PROJECT

Egypt Water impact & Intelligence

Graduation Project: Phase System Analysis & Design

Project Planning • Literature Review • Requirements Gathering
Team Responsibilities • Project Timeline

Team Members:

Aya Khaled
Duha Kamal
Rahma Abdelmuhaymin
Nada Ayman
Nada Elsayed
Niera Elshamy

Supervisor: **Eng / Maged Magdy**

Project Stage: Initial Planning / Repository Setup
GitHub: github.com/nadaxz863/Egypt-Water-impact-Intelligence

Table of Contents

1. Introduction 3
1.1 Problem Statement 3
1.2 Project Objectives 3
1.3 Document Scope and Structure 3
2. System Analysis 4
2.1 Actors 4
2.2 Use Case Diagram 4
2.3 Use Case Descriptions 6
2.4 Detailed Use Case: Upload and Process Dataset 7
2.5 Functional Requirements 7
2.6 Non-Functional Requirements 8
3. Software Architecture 9
3.1 Proposed Architecture Style 9
3.2 Architecture Layers 9
3.3 Proposed Technology Stack 9
3.4 Software Architecture Diagram 10
4. Database Design 11
4.1 ER Diagram (Entity-Relationship Diagram) 11
4.2 Logical Schema 12
4.3 Physical Schema 17
4.4 Normalization Considerations 20
4.5 Database Design Validation Notes 20
5. Data Flow & System Behavior 22
5.1 Context-Level DFD 22
5.2 Level 1 DFD 23
5.3 Sequence Diagram – Dashboard Interaction 25
5.4 Sequence Diagram – Chatbot Interaction 26
5.5 Activity Diagram – System Workflow 28
5.6 State Diagram – Dataset Processing Lifecycle 30
5.7 Class Diagram – System Structure 32
6. Design Notes & Implementation Alignment 35
6.1 Final Review Checklist 35
Appendix A: Source Material Coverage 36

List of Figures

Figure 1: Use Case Diagram – Egypt Water Impact & Intelligence System 5
Figure 2: Software Architecture Diagram (Layered / Modular Architecture) 10
Figure 3: ER Diagram (Entity-Relationship Diagram) – placeholder, not yet created..... 11
Figure 4: Logical Schema – dimensions, bridge table, and fact tables..... 12
Figure 5: Physical Schema – table structure (dbdiagram.io export) 17
Figure 6: Context-Level Data Flow Diagram 22
Figure 7: Level 1 Data Flow Diagram 23
Figure 8: Sequence Diagram – Dashboard Interaction..... 25
Figure 9: Sequence Diagram – Chatbot Interaction 26
Figure 10: Activity Diagram – System Workflow..... 28
Figure 11: State Diagram – Dataset Processing Lifecycle 30
Figure 12: Class Diagram – System Structure 32

1. Introduction

1.1 Problem Statement

Water resources are facing increasing pressure due to climate change, population growth, agricultural demand, and inefficient resource management. Data related to water, agriculture, climate, economy, and electricity is often scattered across multiple sources and stored in different formats, making it difficult to analyze these factors together.

The lack of an integrated platform limits the ability to identify trends, understand relationships between different indicators, monitor water-related challenges, and make data-driven decisions.

Egypt Water Impact & Intelligence aims to address this problem by integrating data from multiple domains into a centralized platform that supports data cleaning, analysis, visualization, predictive modeling, and user interaction through a chatbot.

1.2 Project Objectives

Table 1: Project objectives

Objective	Description
Data Integration	Collect and integrate datasets related to water resources, agriculture, climate, economy, and electricity.
Data Quality	Clean, validate, and transform raw datasets to improve data consistency and reliability.
Centralized Storage	Design and implement a structured SQL database for storing integrated data.
Data Analysis	Analyze historical trends, patterns, and relationships between water-related indicators.
Data Visualization	Develop interactive dashboards to monitor key performance indicators (KPIs).
Predictive Analytics	Apply machine learning models to identify patterns and generate future estimates for selected indicators.
Intelligent Interaction	Provide a chatbot that allows users to ask questions and receive data-based answers.
Decision Support	Present meaningful insights that support better understanding and management of water-related challenges in Egypt.

1.3 Document Scope and Structure

This document consolidates the analysis and design of the Egypt Water Impact & Intelligence system. All diagrams and schemas describe the **proposed design**. Nothing in this document should be read as a statement that a component has already been built or tested unless the team confirms it (see Section 6).

- **Section 2 – System Analysis:** actors, use cases, functional and non-functional requirements.
- **Section 3 – Software Architecture:** architecture style, layers, proposed technology stack, and architecture diagram.

- **Section 4 – Database Design:** ER diagram (placeholder), logical schema, physical schema, normalization considerations, and validation notes.
- **Section 5 – Data Flow & System Behavior:** DFDs, sequence, activity, state, and class diagrams.
- **Section 6 – Design Notes & Implementation Alignment:** alignment rules and the final review checklist.
- **Appendix A – Source Material Coverage:** how each supplied file is represented in this document.

2. System Analysis

2.1 Actors

The use case model identifies the main actors who interact with the system and the operations available to each actor.

Table 2: System actors

Actor	Description
Admin	Manages datasets, monitors data quality, and controls system data operations.
Data Analyst	Explores datasets, performs analysis, monitors KPIs, and generates insights.
End User	Views dashboards, applies filters, and interacts with the chatbot.

2.2 Use Case Diagram

The diagram below shows the use cases available to each actor inside the system boundary, “Egypt Water Impact & Intelligence System”. The following «include» relationships are shown in the diagram:

- Upload Dataset includes Validate Dataset.
- Clean & Transform Data includes Validate Dataset.
- Explore Dashboard includes View KPIs & Visualizations.
- Interact with Chatbot includes Submit Question and Retrieve Relevant Data.

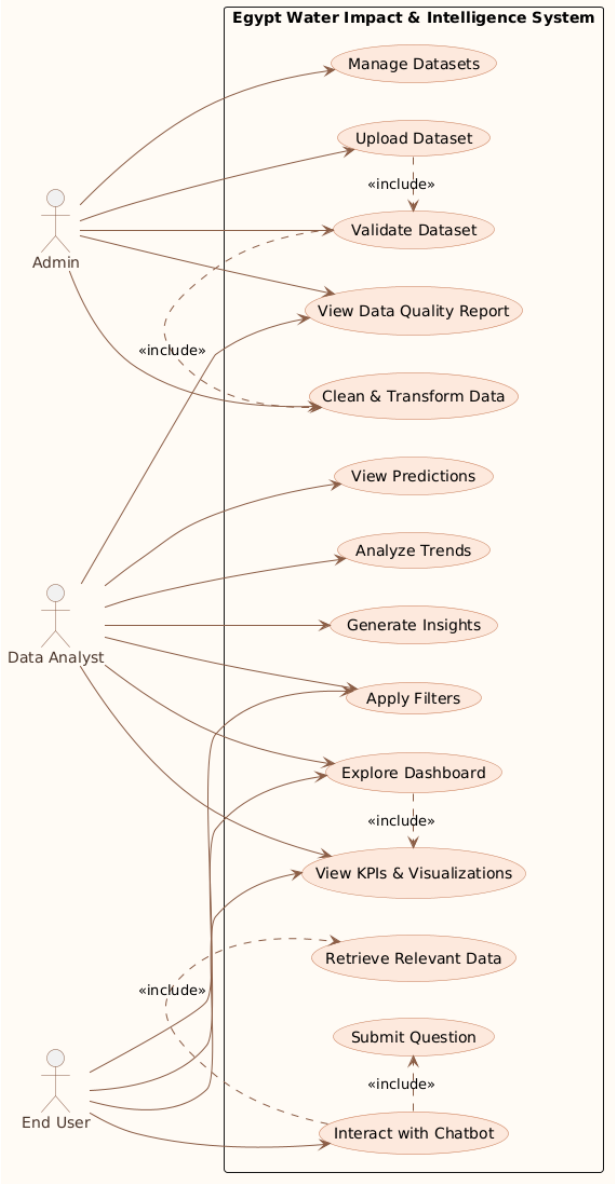


Figure 1: Use Case Diagram – Egypt Water Impact & Intelligence System

2.3 Use Case Descriptions

Table 3: Use case descriptions

Use Case	Actor	Description
Upload Dataset	Admin	Uploads a new dataset to the system.
Validate Dataset	Admin	Checks file structure, missing values, and data validity.
Clean & Transform Data	Admin	Handles missing values, duplicates, and data formatting.
Manage Datasets	Admin	Manages available datasets and their processing status.
View Data Quality Report	Admin, Data Analyst	Reviews data quality issues and validation results.
Explore Dashboard	Data Analyst, End User	Opens dashboards to explore project indicators.
Apply Filters	Data Analyst, End User	Filters results by available dimensions such as year or location.
View KPIs & Visualizations	Data Analyst, End User	Views calculated indicators, charts, and summaries.
Analyze Trends	Data Analyst	Examines historical changes and relationships between indicators.
Generate Insights	Data Analyst	Produces analytical findings based on available data.
Interact with Chatbot	End User	Uses natural-language questions to explore project data.
View Predictions	Data Analyst	Views model outputs for supported indicators.
Submit Question *	End User (via chatbot)	Included use case: the user enters a natural-language question through the chatbot interface.
Retrieve Relevant Data *	System (via chatbot)	Included use case: relevant project data is retrieved from the database to answer the question.

** These two included use cases appear in the Use Case Diagram but were not described in the original table. Their descriptions are taken from the chatbot sequence diagram (Section 5.4) and should be confirmed by the team.*

2.4 Detailed Use Case: Upload and Process Dataset

Table 4: Detailed use case – Upload and Process Dataset

Field	Description
Primary Actor	Admin
Precondition	The Admin has access to the system and a dataset file is available.
Main Flow	1. Admin opens dataset management. 2. Selects and uploads a dataset. 3. System checks file format and structure. 4. System validates the dataset. 5. System identifies missing values, duplicates, and inconsistencies. 6. System performs cleaning and transformation. 7. Processed data is loaded into the central database. 8. Processing status is updated. 9. A data quality report is generated.
Alternative Flow	Invalid format: display an error. Validation failure: report detected issues. Database loading failure: record the error and allow retry.
Postcondition	The dataset is processed and stored, or its errors are reported to the Admin.

2.5 Functional Requirements

Table 5: Functional requirements

ID	Functional Requirement
FR-01	The system shall allow the Admin to upload datasets from supported file formats.
FR-02	The system shall validate uploaded datasets before processing.
FR-03	The system shall detect missing values, duplicate records, and data inconsistencies.
FR-04	The system shall clean and transform datasets before loading them into the database.
FR-05	The system shall store processed data in a centralized SQL database.
FR-06	The system shall generate data quality reports.
FR-07	The system shall provide interactive dashboards for exploring project indicators.
FR-08	The system shall allow users to filter dashboard results using available dimensions.
FR-09	The system shall calculate and display relevant KPIs and analytical metrics.
FR-10	The system shall support historical trend analysis across available datasets.
FR-11	The system shall provide machine learning predictions for selected indicators.
FR-12	The system shall allow users to submit natural-language questions through the chatbot.
FR-13	The chatbot shall retrieve relevant information from the available project data.
FR-14	The chatbot shall generate responses based on retrieved data.
FR-15	The system shall display analytical insights and prediction results to authorized users.

2.6 Non-Functional Requirements

Table 6: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Dashboard queries should return results within an acceptable response time under normal usage.
NFR-02	Security	Access to administrative operations shall be restricted to authorized users.
NFR-03	Reliability	The system should handle invalid datasets and processing errors without corrupting stored data.
NFR-04	Usability	The user interface should be clear, simple, and easy to navigate.
NFR-05	Scalability	The system architecture should support adding new datasets and analytical modules.
NFR-06	Maintainability	The system components should be modular and easy to update or maintain.
NFR-07	Data Integrity	Data validation rules should help maintain consistency and accuracy during processing.
NFR-08	Compatibility	The system should support commonly used web browsers and supported dataset formats.
NFR-09	Availability	The system should be accessible whenever the hosting environment and required services are operational.
NFR-10	Explainability	Prediction and chatbot outputs should provide understandable context about results and data sources where possible.

3. Software Architecture

3.1 Proposed Architecture Style

The proposed design uses a **Layered (Modular) Architecture**. The system is divided into layers, with each layer responsible for a specific group of operations. This approach keeps responsibilities separated and is more practical for the current project scope than starting with a microservices architecture.

3.2 Architecture Layers

Table 7: Architecture layers, components, and responsibilities

Layer	Main Components	Responsibilities
Presentation Layer	Interactive Dashboard, Chatbot Interface, Admin Interface	Displays information and receives user requests.
Application Layer	Backend API, Request Management, Authentication & Access Control	Handles requests, coordinates services, and controls access.
Data Processing Layer	Data Collection, Validation, Cleaning, Transformation, ETL	Prepares and integrates data from different sources.
Analytics & AI Layer	KPI Calculation, Statistical Analysis, ML Models, Chatbot Processing	Analyzes data, generates predictions, and answers questions.
Data Layer	SQL Server Database, Processed Data, Analysis Results	Stores and manages datasets and analytical outputs.

3.3 Proposed Technology Stack

Table 8: Proposed technology stack

Component	Proposed Technology
Database	SQL Server
Data Processing	Python, Pandas
Data Analysis	Python, SQL
Data Visualization	Power BI
Machine Learning	Scikit-learn
Backend API	Python, FastAPI
Chatbot	LLM API + Python
Development Environment	VS Code, Jupyter Notebook

Note: These technologies represent the proposed stack. The final documentation should be updated to match the tools and components actually selected and implemented by the team.

3.4 Software Architecture Diagram

The diagram shows the five layers of the proposed architecture and the data flow between them: external data sources feed the Data Processing Layer, which loads the Data Layer; the Analytics & AI Layer reads from and writes to the Data Layer; the Application Layer (Backend API) coordinates requests between the Analytics & AI Layer, the Data Layer, and the Presentation Layer.

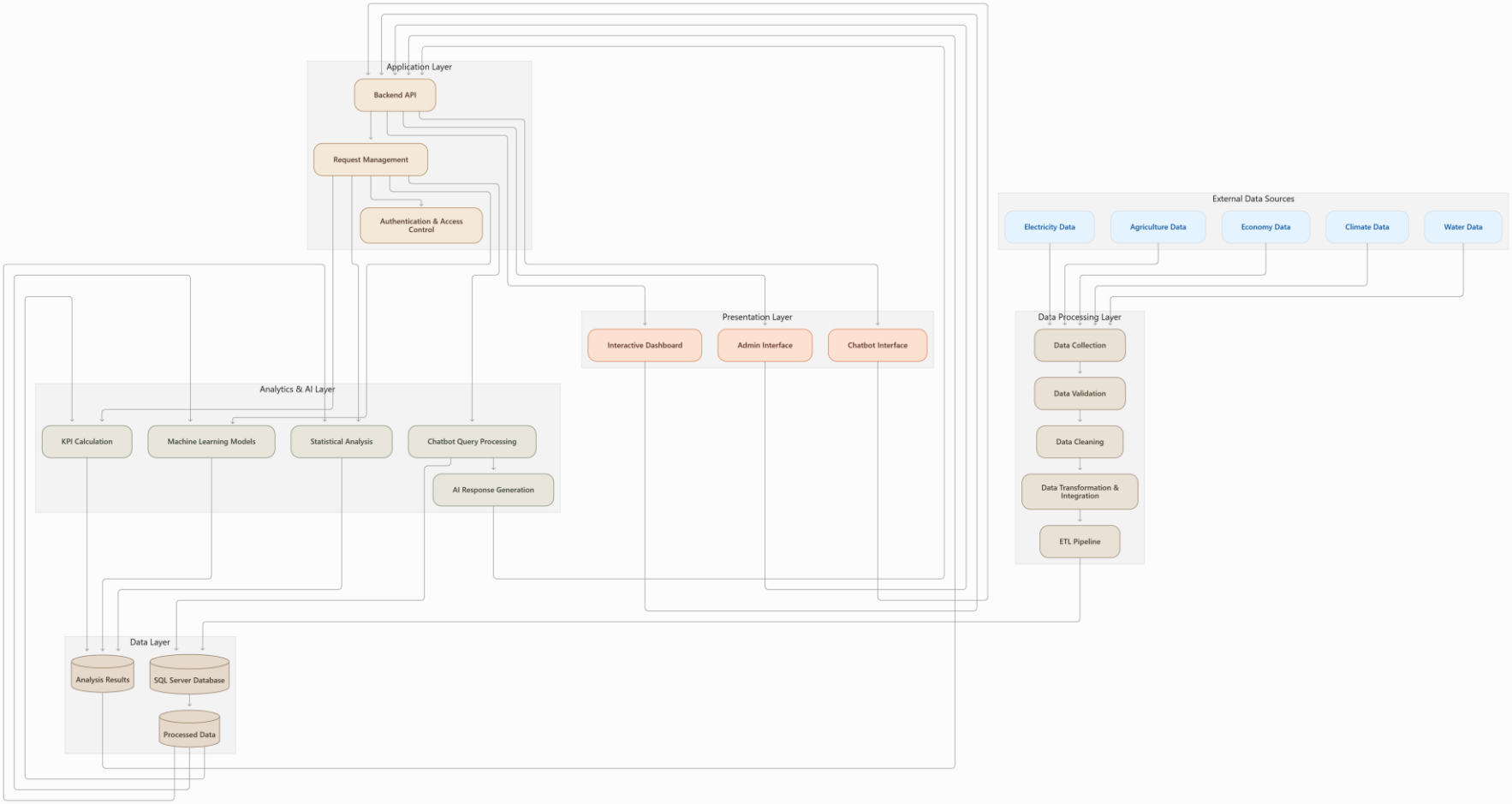


Figure 2: Software Architecture Diagram (Layered / Modular Architecture)

4. Database Design

This section documents the database design at three levels: the conceptual ER diagram, the logical schema, and the physical schema. The logical schema and physical schema diagrams were supplied and are included below. The ER diagram has not been supplied yet, so a reserved placeholder is provided for it.

4.1 ERD (Entity-Relationship Diagram)

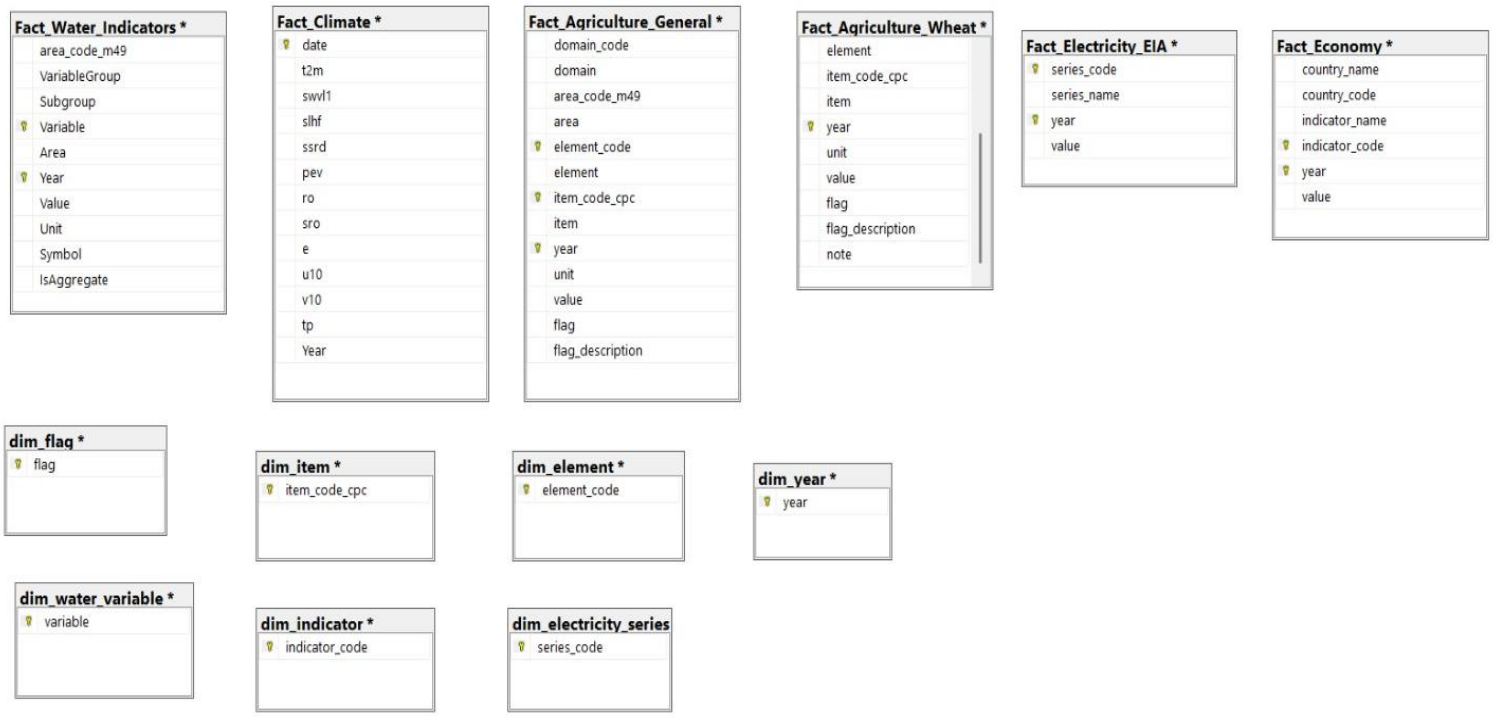


Figure 3: ER Diagram (Entity-Relationship Diagram) – placeholder, not yet created

4.2 Logical Schema

The logical schema follows a dimensional (star-style) design: descriptive attributes are stored in dimension tables and measurable observations in fact tables. It contains 7 dimension tables and 6 fact tables. The diagram is embedded at high resolution; zoom in to read table and column details. The full column list is reproduced in the tables that follow.

Figure 4: Logical Schema – dimensions, bridge table, and fact tables

4.2.1 Dimension and Bridge Tables

Table 10 details the logical specifications for all 7 dimension tables.

Table Name	Column Name	Data Type	Key Constraint / References
dim_year	year	int64	Primary Key (PK)
dim_item	item_code_cpc	int64	Primary Key (PK)
dim_item	item	object	Attribute

Table Name	Column Name	Data Type	Key Constraint / References
dim_flag	flag	object	Primary Key (PK)
dim_flag	flag_description	object	Attribute
dim_element	element_code	int64	Primary Key (PK)
dim_element	element	object	Attribute
dim_element	unit	object	Attribute
dim_electricity_series	series_code	object	Primary Key (PK)
dim_electricity_series	series_name	object	Attribute
dim_electricity_series	metric	object	Attribute
dim_electricity_series	unit	object	Attribute
dim_indicator	indicator_code	object	Primary Key (PK)
dim_indicator	indicator_name	object	Attribute
dim_water_variable	variable	object	Primary Key (PK)
dim_water_variable	variable_group	object	Attribute
dim_water_variable	subgroup	object	Attribute
dim_water_variable	unit	object	Attribute

4.2.2 Fact Tables

Table 11 details the logical specifications for all 6 fact tables.

Fact Table Name	Column Name	Data Type	Key Constraint / References
Fact_Agriculture_General	domain_code	object	Attribute
Fact_Agriculture_General	domain	object	Attribute
Fact_Agriculture_General	area_code_m49	int64	Attribute
Fact_Agriculture_General	area	object	Attribute
Fact_Agriculture_General	element_code	int64	Composite PK / FK -> dim_element.element_code
Fact_Agriculture_General	element	object	Attribute
Fact_Agriculture_General	item_code_cpc	int64	Composite PK / FK -> dim_item.item_code_cpc
Fact_Agriculture_General	item	object	Attribute
Fact_Agriculture_General	year	int64	Composite PK / FK -> dim_year.year
Fact_Agriculture_General	unit	object	Attribute
Fact_Agriculture_General	value	float64	Attribute
Fact_Agriculture_General	flag	object	FK -> dim_flag.flag
Fact_Agriculture_General	flag_description	object	Attribute
Fact_Agriculture_Wheat	domain_code	object	Attribute
Fact_Agriculture_Wheat	domain	object	Attribute
Fact_Agriculture_Wheat	area_code_m49	int64	Attribute
Fact_Agriculture_Wheat	area	object	Attribute
Fact_Agriculture_Wheat	element_code	int64	FK -> dim_element.element_code
Fact_Agriculture_Wheat	element	object	Attribute
Fact_Agriculture_Wheat	item_code_cpc	int64	FK -> dim_item.item_code_cpc
Fact_Agriculture_Wheat	item	object	Attribute
Fact_Agriculture_Wheat	year_code	int64	Attribute
Fact_Agriculture_Wheat	year	int64	PK / FK -> dim_year.year
Fact_Agriculture_Wheat	unit	object	Attribute
Fact_Agriculture_Wheat	value	float64	Attribute
Fact_Agriculture_Wheat	flag	object	FK -> dim_flag.flag
Fact_Agriculture_Wheat	flag_description	object	Attribute
Fact_Agriculture_Wheat	note	object	Attribute
Fact_Climate	date	datetime64[ns]	Primary Key (PK)
Fact_Climate	t2m	float64	Attribute
Fact_Climate	swvl1	float64	Attribute
Fact_Climate	slhf	float64	Attribute
Fact_Climate	ssrd	float64	Attribute
Fact_Climate	pev	float64	Attribute
Fact_Climate	ro	float64	Attribute
Fact_Climate	sro	float64	Attribute
Fact_Climate	e	float64	Attribute
Fact_Climate	u10	float64	Attribute
Fact_Climate	v10	float64	Attribute
Fact_Climate	tp	float64	Attribute
Fact_Climate	Year	int64	FK -> dim_year.year
Fact_Electricity_EIA	series_code	object	Composite PK / FK -> dim_electricity_series.series_code
Fact_Electricity_EIA	series_name	object	Attribute
Fact_Electricity_EIA	year	int64	Composite PK / FK -> dim_year.year
Fact_Electricity_EIA	value	float64	Attribute
Fact_Economy	country_name	object	Attribute
Fact_Economy	country_code	object	Attribute

Fact Table Name	Column Name	Data Type	Key Constraint / References
Fact_Economy	indicator_name	object	Attribute
Fact_Economy	indicator_code	object	Composite PK / FK -> dim_indicator.indicator_code
Fact_Economy	year	int64	Composite PK / FK -> dim_year.year
Fact_Economy	value	float64	Attribute
Fact_Water_Indicators	area_code_m49	int64	Attribute
Fact_Water_Indicators	VariableGroup	object	Attribute
Fact_Water_Indicators	Subgroup	object	Attribute
Fact_Water_Indicators	Variable	object	Composite PK / FK -> dim_water_variable.variable
Fact_Water_Indicators	Area	object	Attribute
Fact_Water_Indicators	Year	int64	Composite PK / FK -> dim_year.year
Fact_Water_Indicators	Value	float64	Attribute
Fact_Water_Indicators	Unit	object	Attribute
Fact_Water_Indicators	Symbol	object	Attribute
Fact_Water_Indicators	IsAggregate	bool	Attribute

4.3 Physical Schema

The physical schema diagram (exported from dbdiagram.io) shows 13 tables with their data types and primary keys. The diagram shows the table structures and primary-key markers; no relationships between the tables are drawn. The diagram is a design artifact; per the validation notes in Section 4.5, the physical SQL schema must be executed and tested before it can be described as implemented.

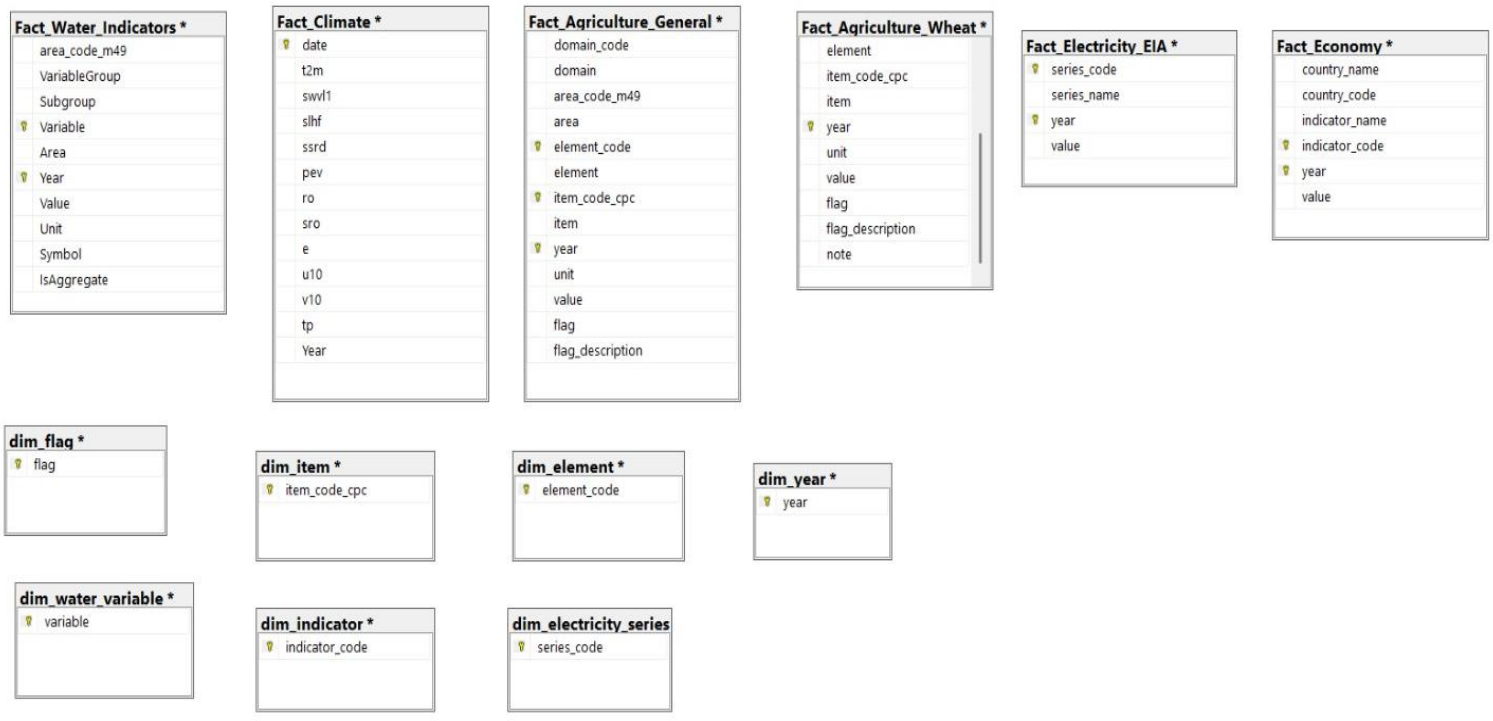


Figure 5: Physical Schema – table structure (dbdiagram.io export)

Table 12 details the physical DDL specifications for all database tables.

Physical Table Name	Column Name	Data Type	Constraint / Key / Nullability
dim_year	year	int64	PRIMARY KEY, NOT NULL
dim_item	item_code_cpc	int64	PRIMARY KEY, NOT NULL

Physical Table Name	Column Name	Data Type	Constraint / Key / Nullability
dim_flag	flag	object	PRIMARY KEY, NOT NULL
dim_element	element_code	int64	PRIMARY KEY, NOT NULL
dim_electricity_series	series_code	object	PRIMARY KEY, NOT NULL
dim_indicator	indicator_code	object	PRIMARY KEY, NOT NULL
dim_water_variable	variable	object	PRIMARY KEY, NOT NULL
Fact_Water_Indicators	area_code_m49	int64	NULLable
Fact_Water_Indicators	VariableGroup	object	NULLable
Fact_Water_Indicators	Subgroup	object	NULLable
Fact_Water_Indicators	Variable	object	PRIMARY KEY, NOT NULL
Fact_Water_Indicators	Area	object	NULLable
Fact_Water_Indicators	Year	int64	PRIMARY KEY, NOT NULL
Fact_Water_Indicators	Value	float64	NULLable
Fact_Water_Indicators	Unit	object	NULLable
Fact_Water_Indicators	Symbol	object	NULLable
Fact_Water_Indicators	IsAggregate	bool	NULLable
Fact_Climate	date	datetime64[ns]	PRIMARY KEY, NOT NULL
Fact_Climate	t2m	float64	NULLable
Fact_Climate	swvl1	float64	NULLable
Fact_Climate	slhf	float64	NULLable
Fact_Climate	ssrd	float64	NULLable
Fact_Climate	pev	float64	NULLable
Fact_Climate	ro	float64	NULLable
Fact_Climate	sro	float64	NULLable
Fact_Climate	e	float64	NULLable
Fact_Climate	u10	float64	NULLable
Fact_Climate	v10	float64	NULLable
Fact_Climate	tp	float64	NULLable
Fact_Climate	Year	int64	NULLable
Fact_Agriculture_General	domain_code	object	NULLable
Fact_Agriculture_General	domain	object	NULLable
Fact_Agriculture_General	area_code_m49	int64	NULLable
Fact_Agriculture_General	area	object	NULLable
Fact_Agriculture_General	element_code	int64	PRIMARY KEY (composite), NOT NULL
Fact_Agriculture_General	element	object	NULLable
Fact_Agriculture_General	item_code_cpc	int64	PRIMARY KEY (composite), NOT NULL
Fact_Agriculture_General	item	object	NULLable
Fact_Agriculture_General	year	int64	PRIMARY KEY (composite), NOT NULL
Fact_Agriculture_General	unit	object	NULLable
Fact_Agriculture_General	value	float64	NULLable
Fact_Agriculture_General	flag	object	NULLable
Fact_Agriculture_General	flag_description	object	NULLable
Fact_Agriculture_Wheat	domain_code	object	NULLable
Fact_Agriculture_Wheat	domain	object	NULLable
Fact_Agriculture_Wheat	area_code_m49	int64	NULLable
Fact_Agriculture_Wheat	area	object	NULLable
Fact_Agriculture_Wheat	element_code	int64	NULLable
Fact_Agriculture_Wheat	element	object	NULLable
Fact_Agriculture_Wheat	item_code_cpc	int64	NULLable
Fact_Agriculture_Wheat	item	object	NULLable
Fact_Agriculture_Wheat	year_code	int64	NULLable
Fact_Agriculture_Wheat	year	int64	PRIMARY KEY, NOT NULL
Fact_Agriculture_Wheat	unit	object	NULLable
Fact_Agriculture_Wheat	value	float64	NULLable
Fact_Agriculture_Wheat	flag	object	NULLable
Fact_Agriculture_Wheat	flag_description	object	NULLable
Fact_Agriculture_Wheat	note	object	NULLable
Fact_Electricity_EIA	series_code	object	PRIMARY KEY (composite), NOT NULL
Fact_Electricity_EIA	series_name	object	NULLable
Fact_Electricity_EIA	year	int64	PRIMARY KEY (composite), NOT NULL
Fact_Electricity_EIA	value	float64	NULLable
Fact_Economy	country_name	object	NULLable
Fact_Economy	country_code	object	NULLable

Physical Table Name	Column Name	Data Type	Constraint / Key / Nullability
Fact_Economy	indicator_name	object	NULLable
Fact_Economy	indicator_code	object	PRIMARY KEY (composite), NOT NULL
Fact_Economy	year	int64	PRIMARY KEY (composite), NOT NULL
Fact_Economy	value	float64	NULLable

4.4 Normalization Considerations

The database design separates descriptive attributes into dimension tables and measurable observations into fact tables. This reduces repeated descriptive data and supports consistent analysis across subject areas.

- **First Normal Form (1NF):** Each field should contain a single atomic value, and repeating groups should be avoided.
- **Second Normal Form (2NF):** Non-key attributes should depend on the whole candidate key, not only part of a composite key.
- **Third Normal Form (3NF):** Non-key attributes should depend on the key, the whole key, and nothing but the key; transitive dependencies should be avoided.
- Surrogate primary keys may be used for dimension and fact tables, while natural/business keys should be retained where needed for source mapping and duplicate checks.
- Unique constraints on each fact table's business grain should be added only after the grain and source-data uniqueness have been validated.

4.5 Database Design Validation Notes

- Climate data currently has no explicit location field, so it should not be linked to Geography unless a reliable location mapping is obtained.
- M49 codes and ISO country codes are different coding systems and require an explicit mapping; they should not be treated as interchangeable.
- Electricity capacity and export source tables do not currently contain a country identifier, so geographic relationships should not be assumed.
- Agriculture_Wheat may overlap with Agriculture_Crops. Check the source records before combining or treating them as independent datasets.
- The grain of each fact table must be documented. For example, the combination of geography, year, item, element, and unit may define an agriculture observation.
- The physical SQL schema must be executed and tested before it can be described as implemented.

5. Data Flow & System Behavior

This section describes how data enters the system, moves through processing and storage, and is used by analytics, dashboards, and the chatbot. Each diagram is followed by a text description of its content.

5.1 Context-Level DFD

The context-level data flow diagram shows the system as a single process interacting with three external entities.

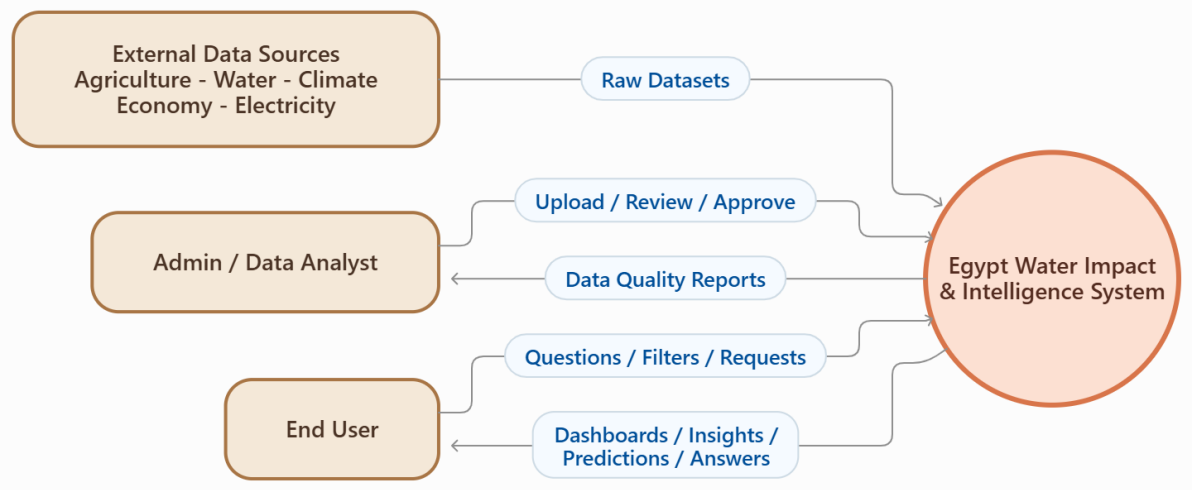


Figure 6: Context-Level Data Flow Diagram

Table 12: Context-level data flows

From	Data Flow	To
External Data Sources (Agriculture, Water, Climate, Economy, Electricity)	Raw Datasets	Egypt Water Impact & Intelligence System
Egypt Water Impact & Intelligence System	Data Quality Reports	Admin / Data Analyst
Admin / Data Analyst	Upload / Review / Approve	Egypt Water Impact & Intelligence System
End User	Questions / Filters / Requests	Egypt Water Impact & Intelligence System
Egypt Water Impact & Intelligence System	Dashboards / Insights / Predictions / Answers	End User

5.2 Level 1 DFD

The Level 1 DFD decomposes the system into six processes and four data stores.

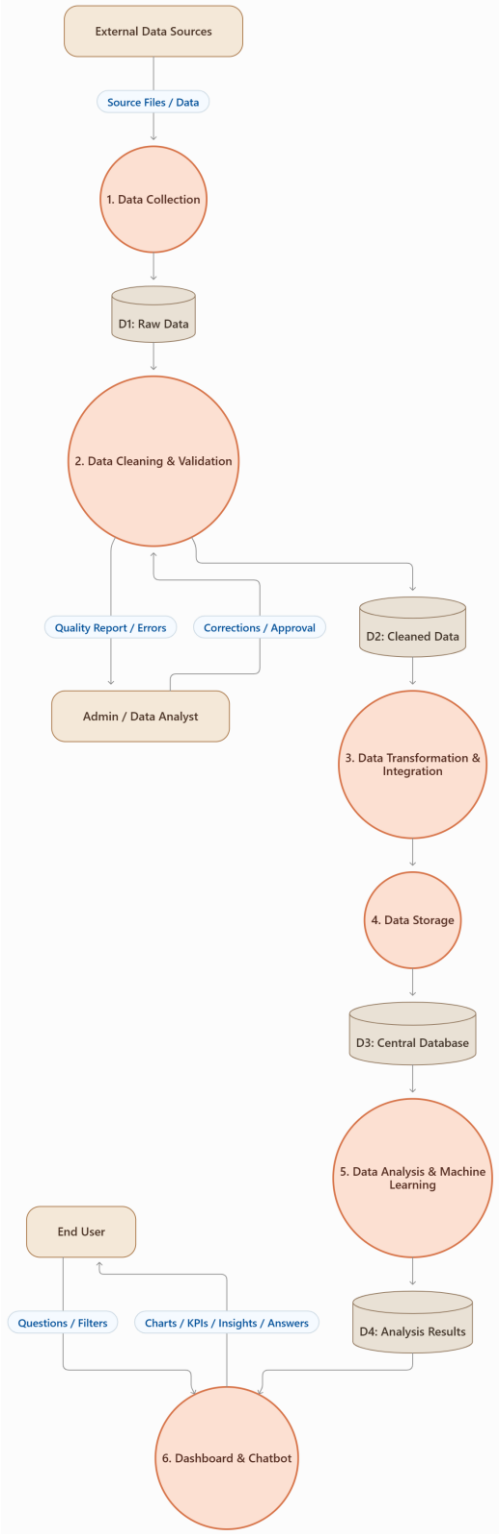


Figure 7: Level 1 Data Flow Diagram

Table 13: Level 1 DFD – processes and data stores

Element	Type	Description / Flow
External Data Sources	External entity	Sends source files / data to Process 1.
Admin / Data Analyst	External entity	Receives quality reports / errors from Process 2 and returns corrections / approval to Process 2.
End User	External entity	Sends questions / filters to Process 6 and receives charts / KPIs / insights / answers.
1. Data Collection	Process	Collects source data and writes it to D1: Raw Data.
2. Data Cleaning & Validation	Process	Reads D1: Raw Data; reports quality issues and errors to the Admin / Data Analyst; receives corrections / approval; writes D2: Cleaned Data.
3. Data Transformation & Integration	Process	Reads D2: Cleaned Data and passes the result to Process 4.
4. Data Storage	Process	Stores integrated data in D3: Central Database.
5. Data Analysis & Machine Learning	Process	Reads D3: Central Database and writes D4: Analysis Results.
6. Dashboard & Chatbot	Process	Reads D4: Analysis Results; serves charts, KPIs, insights, and answers to the End User.
D1: Raw Data	Data store	Raw collected data.
D2: Cleaned Data	Data store	Validated and cleaned data.
D3: Central Database	Data store	Integrated data in the central database.
D4: Analysis Results	Data store	Outputs of analysis and machine learning.

5.3 Sequence Diagram – Dashboard Interaction

The sequence below shows how a user opens the dashboard and then applies filters.

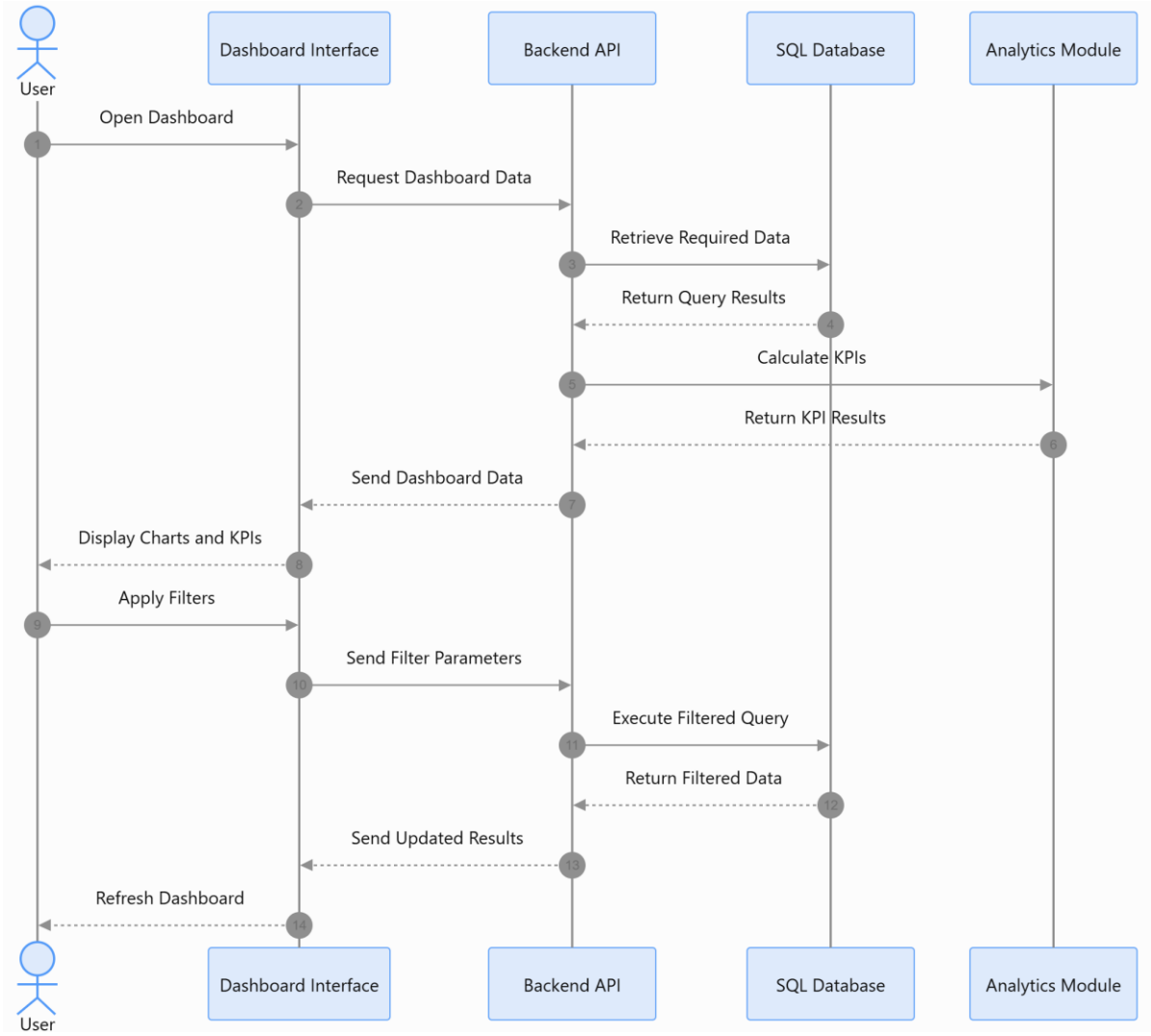


Figure 8: Sequence Diagram – Dashboard Interaction

Table 14: Dashboard interaction – message sequence

Step	From	To	Message
1	User	Dashboard Interface	Open Dashboard
2	Dashboard Interface	Backend API	Request Dashboard Data
3	Backend API	SQL Database	Retrieve Required Data
4	SQL Database	Backend API	Return Query Results
5	Backend API	Analytics Module	Calculate KPIs
6	Analytics Module	Backend API	Return KPI Results
7	Backend API	Dashboard Interface	Send Dashboard Data
8	Dashboard Interface	User	Display Charts and KPIs
9	User	Dashboard Interface	Apply Filters
10	Dashboard Interface	Backend API	Send Filter Parameters
11	Backend API	SQL Database	Execute Filtered Query
12	SQL Database	Backend API	Return Filtered Data
13	Backend API	Dashboard Interface	Send Updated Results
14	Dashboard Interface	User	Refresh Dashboard

5.4 Sequence Diagram – Chatbot Interaction

The sequence below shows how a user question is processed and answered using data retrieved from the database.

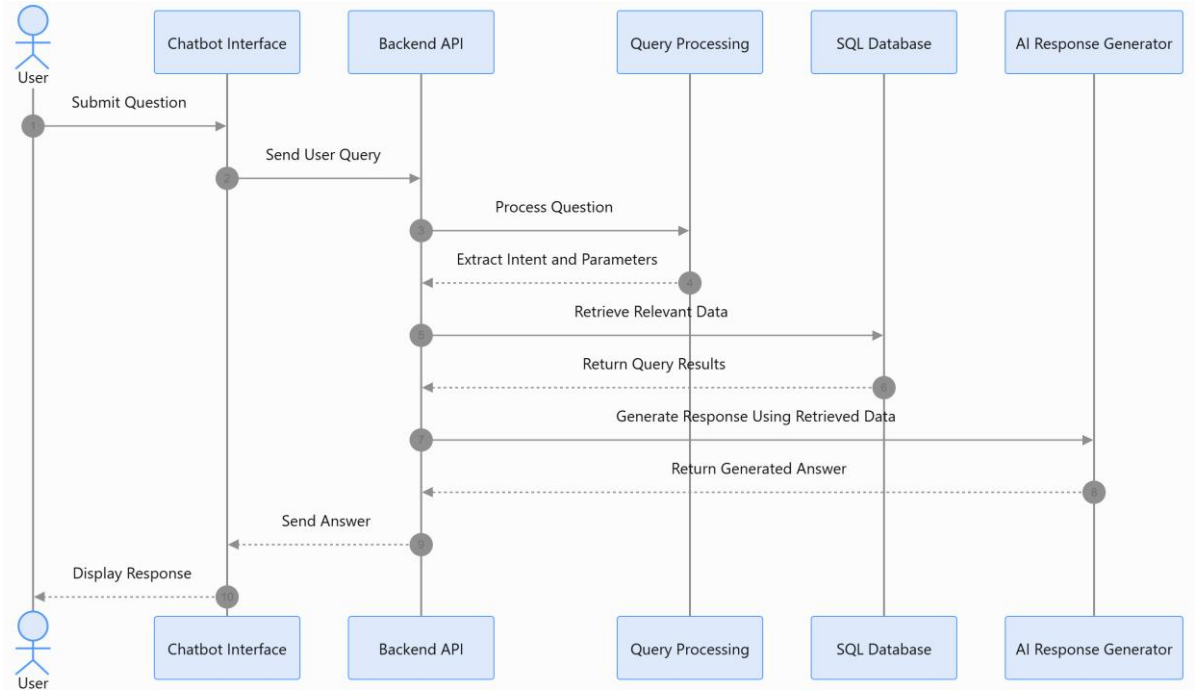


Figure 9: Sequence Diagram – Chatbot Interaction

Table 15: Chatbot interaction – message sequence

Step	From	To	Message
1	User	Chatbot Interface	Submit Question
2	Chatbot Interface	Backend API	Send User Query
3	Backend API	Query Processing	Process Question
4	Query Processing	Backend API	Extract Intent and Parameters
5	Backend API	SQL Database	Retrieve Relevant Data
6	SQL Database	Backend API	Return Query Results
7	Backend API	AI Response Generator	Generate Response Using Retrieved Data
8	AI Response Generator	Backend API	Return Generated Answer
9	Backend API	Chatbot Interface	Send Answer
10	Chatbot Interface	User	Display Response

5.5 Activity Diagram – System Workflow

The activity diagram describes the overall workflow: the user chooses either the Dashboard or the Chatbot service, completes the request, and may then make another request.

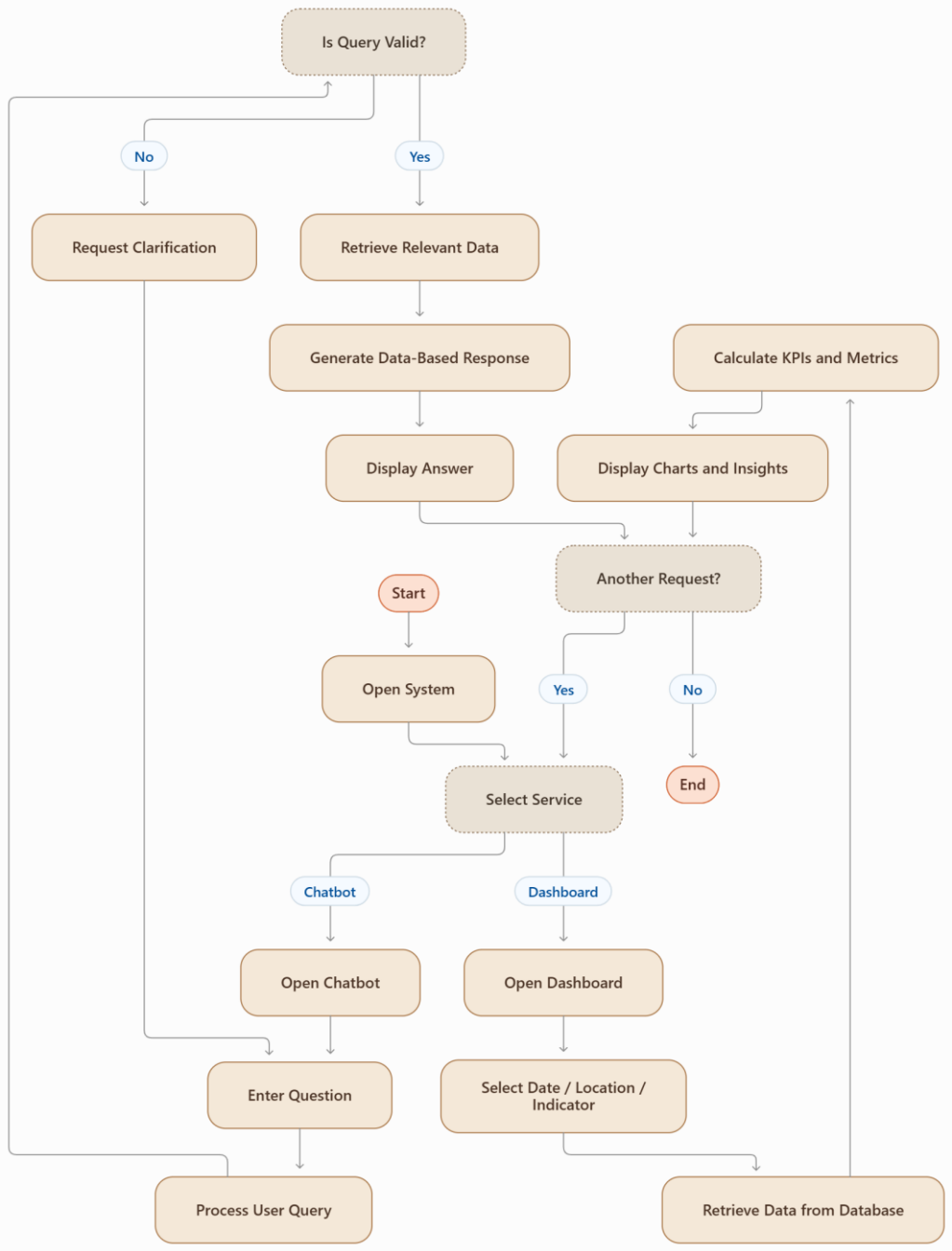


Figure 10: Activity Diagram – System Workflow

Table 16: System workflow – activity flow

Flow	Activities
Common start	Start → Open System → Select Service (decision: Dashboard or Chatbot).
Dashboard branch	1. Open Dashboard 2. Select Date / Location / Indicator 3. Retrieve Data from Database 4. Calculate KPIs and Metrics 5. Display Charts and Insights
Chatbot branch	1. Open Chatbot 2. Enter Question 3. Process User Query 4. Decision – Is Query Valid? No: Request Clarification, then return to Enter Question. Yes: Retrieve Relevant Data → Generate Data-Based Response → Display Answer.
Common end	After the dashboard branch or the chatbot branch: decision – Another Request? Yes: return to Select Service. No: End.

5.6 State Diagram – Dataset Processing Lifecycle

The state diagram shows the lifecycle of a dataset from upload to publication, including the failure and retry paths for validation and loading.

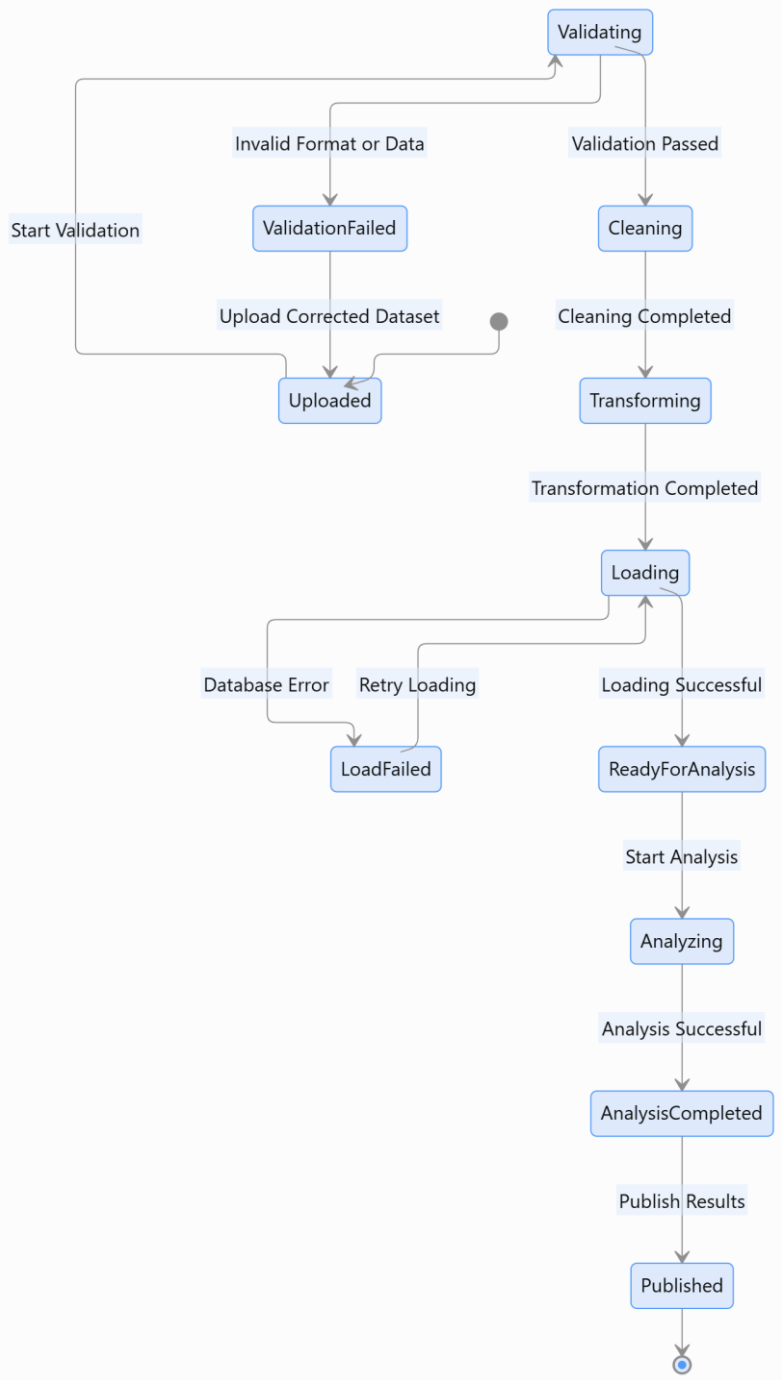


Figure 11: State Diagram – Dataset Processing Lifecycle

Table 17: Dataset lifecycle – state transitions

From State	Event	To State
Initial state	Dataset is submitted	Uploaded
Uploaded	Start Validation	Validating
Validating	Invalid Format or Data	ValidationFailed
Validating	Validation Passed	Cleaning
ValidationFailed	Upload Corrected Dataset	Uploaded
Cleaning	Cleaning Completed	Transforming
Transforming	Transformation Completed	Loading
Loading	Database Error	LoadFailed
LoadFailed	Retry Loading	Loading
Loading	Loading Successful	ReadyForAnalysis
ReadyForAnalysis	Start Analysis	Analyzing
Analyzing	Analysis Successful	AnalysisCompleted
AnalysisCompleted	Publish Results	Published
Published	—	Final state

5.7 Class Diagram – System Structure

The class diagram shows the main classes of the system, their attributes and operations, and the relationships between them. Admin and DataAnalyst are specializations of User.

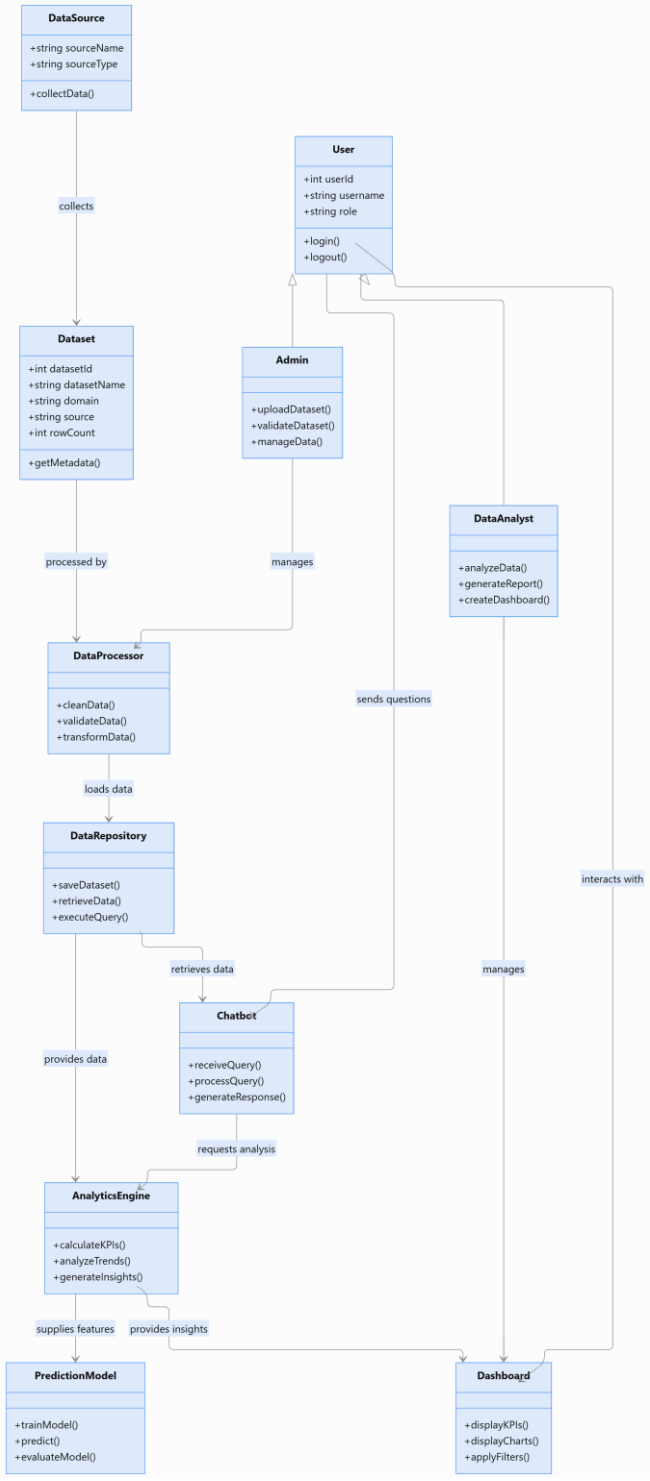


Figure 12: Class Diagram – System Structure

Table 18: Class diagram – classes, attributes, and operations

Class	Attributes	Operations
User	userId (int) username (string) role (string)	login logout
Admin (extends User)	—	uploadDataset validateDataset manageData
DataAnalyst (extends User)	—	analyzeData generateReport createDashboard
Dataset	datasetId (int) datasetName (string) domain (string) source (string) rowCount (int)	getMetadata
DataSource	sourceName (string) sourceType (string)	collectData
DataProcessor	—	cleanData validateData transformData
DataRepository	—	saveDataset retrieveData executeQuery
AnalyticsEngine	—	calculateKPIs analyzeTrends generateInsights
PredictionModel	—	trainModel predict evaluateModel
Dashboard	—	displayKPIs displayCharts applyFilters
Chatbot	—	receiveQuery processQuery generateResponse

Table 19: Class diagram – relationships

From	Relationship	To
Admin	Inheritance (is a)	User
DataAnalyst	Inheritance (is a)	User
DataSource	collects	Dataset
Dataset	processed by	DataProcessor
DataProcessor	loads data	DataRepository
DataRepository	provides data	AnalyticsEngine
AnalyticsEngine	supplies features	PredictionModel
AnalyticsEngine	provides insights	Dashboard
DataRepository	retrieves data	Chatbot
Chatbot	requests analysis	AnalyticsEngine
User	interacts with	Dashboard
User	sends questions	Chatbot
Admin	manages	DataProcessor
DataAnalyst	manages	Dashboard

6. Design Notes & Implementation Alignment

- All diagrams in this document represent the proposed system design. Update them if the implemented architecture changes.
- Do not document a backend API, chatbot LLM integration, authentication mechanism, or ML deployment as implemented until the team has built and tested it.
- If Power BI is connected directly to SQL Server, the architecture should show that actual connection rather than an assumed separate dashboard frontend.
- Keep the same actor names, component names, database entities, and data-flow terminology consistent across all diagrams.
- The ERD, Logical Schema, Physical Schema, and Use Case Diagram placeholders must be replaced with final exported diagrams before submission.
- Validate source mappings, fact-table grain, keys, and constraints before declaring the database design complete.

7. UI&UX Design




Egypt Water
Impact & Intelligence
Overview
Water
Agriculture
Energy
Climate
Economy
Trade
AI & Insights
Water Availability
Agricultural Output
Electricity Generated
Climate Index
Economic Impact
Water Resources Overview
Impact on Agriculture
Energy & Climate Trends
Economic & Trade Insights


Egypt Water
Impact & Intelligence
Overview
Water
Agriculture
Energy
Climate
Economy
Trade
AI & Insights
Water Availability
Agricultural Output
Electricity Generated
Climate Index
Economic Impact
Water Resources Overview
Impact on Agriculture
Energy & Climate Trends
Economic & Trade Insights


Egypt Water
Impact & Intelligence
Overview
Water
Agriculture
Energy
Climate
Economy
Trade
AI & Insights


Egypt Water
Impact & Intelligence
Overview
Water
Agriculture
Energy
Climate
Economy
Trade
AI & Insights


Egypt Water
Impact & Intelligence
Overview
Water
Agriculture
Energy
Climate
Economy
Trade
AI & Insights

8. Proposed System Architecture and Implementation Plan

This section defines a proposed technical architecture and implementation plan for Egypt Water Impact & Intelligence. It is based on the project objectives, actors, functional and non-functional requirements, and dimensional database design described in the preceding sections. The technologies listed below are planned choices; the backend, frontend, database connection, SQL scripts, API endpoints, tests, and deployment have not yet been implemented or executed.
Status convention: “Proposed” means a design recommendation for future implementation. No implementation or test completion is implied by the diagrams, endpoint examples, or test cases in this section.

8.1 Technology Stack

Layer / Technology	Proposed role	Planned interaction
Frontend — Streamlit	Browser-based interactive presentation layer for dashboards, filters, KPI views, data-quality summaries, and the chatbot interface where that feature is implemented.	Collects user input and displays results; sends HTTP requests to the FastAPI service. It should not connect directly to SQL Server.
Backend — FastAPI (Python)	Application/API layer responsible for REST routes, request validation, orchestration of processing and analytics services, and consistent JSON responses.	Receives requests from Streamlit, invokes Python service modules, and accesses SQL Server through a dedicated data-access layer.
Database — Microsoft SQL Server	Central relational store for the planned dimensional model, including dimension and fact tables and approved processed/analytical data.	Receives parameterized SQL operations from the backend data-access layer; returns rows/aggregates for processing and presentation.

The proposed request path is User Browser → Streamlit → FastAPI → Python service modules → SQL Server. Results return through the same path in reverse. Dataset ingestion and batch preparation may also be run as controlled Python jobs; those jobs should use the same validation rules and database access conventions as the API.
Only the three technologies specified as planned choices are treated as confirmed technology selections here. Specific Python packages, database drivers, authentication providers, hosting vendors, and external AI/ML services remain implementation decisions and are not asserted as currently used.

8.2 Proposed Deployment Diagram

Figure 13 presents a logical UML deployment view. Hardware and execution environments are shown as hosting-node roles rather than existing servers: the actual hosting provider, machine sizes, network topology, and deployment configuration have not yet been selected.

Figure 13: Proposed UML Deployment Diagram — Egypt Water Impact & Intelligence
Explanation: Users access the Streamlit application from a web browser over HTTPS. Streamlit communicates with the FastAPI application using HTTPS requests and JSON payloads. FastAPI validates and routes each request to the appropriate Python service. Services retrieve or persist data through a controlled repository/data-access layer using a secured SQL connection to Microsoft SQL Server. The database is not exposed directly to end-user browsers. In a production deployment, the frontend and API should be reachable

through controlled public endpoints, while SQL Server should remain on a private network or managed private endpoint.
The diagram is a target-state proposal, not evidence of deployed nodes, live communication links, or configured network security.

8.3 Proposed Component Diagram

Figure 14 decomposes the target application into components and their dependencies. The service boundaries are proposed logical modules derived from the documented use cases and requirements; their internal code structure will be finalized during implementation.

Figure 14: Proposed UML Component Diagram — Egypt Water Impact & Intelligence

Component	Responsibility / interface	Dependencies
Streamlit UI	Dashboard, dimension filters, KPI and trend presentation, data-quality views, and user-facing chatbot screen if delivered.	Calls the API client layer; renders API results.
API client layer	Centralizes HTTP calls, request payloads, timeouts, and display-safe error handling.	Streamlit UI; FastAPI REST interface.
FastAPI routes and schemas	Defines route contracts, validates incoming data, invokes services, and serializes responses.	Python service modules; repository/data-access interface.
Dataset service	Proposed upload metadata handling, file/structure validation, cleaning and transformation orchestration, and processing status.	API layer; repository; approved input formats and validation rules.
Analytics service	Proposed KPI aggregation, historical trend retrieval, and an extension point for prediction models if selected.	API layer; repository; validated analytical definitions.
Chatbot service (conditional)	Proposed question handling and retrieval of relevant project data; answer generation requires a separately selected and approved approach.	API layer; repository; optional external model/provider only if adopted.
Data access / repository	Owns database connection configuration, parameterized queries, transaction boundaries, and mapping between database rows and service data.	Python services; Microsoft SQL Server.
Microsoft SQL Server	Stores the planned dimensions, fact tables, and approved processed or analytical results.	Accessed only through the repository/data-access component.

The chatbot and prediction modules are included as conditional extension points because the existing requirements describe intelligent interaction and predictive analytics, but the detailed model/provider and operational design are not specified. They should only be treated as delivered features after their implementation and validation.

8.4 Proposed API Documentation

No FastAPI application or API endpoints currently exist. The following REST contract is a suggested starting specification derived from the documented use cases: dataset upload and validation, data-quality reporting, dashboard exploration, trend analysis, and chatbot interaction. Route names, field names, status codes, and response schemas are proposals and may be revised during implementation.

Endpoint name	Method and proposed route	Purpose

Service health	GET /api/v1/health	Proposed readiness check for the API process and (optionally) database dependency.
Register dataset upload	POST /api/v1/datasets	Accept dataset metadata and an uploaded file; create a processing record.
Validate dataset	POST /api/v1/datasets/{dataset_id}/validate	Run structural and data-quality checks and return validation findings.
Process dataset	POST /api/v1/datasets/{dataset_id}/process	Start or request cleaning, transformation, and loading for a validated dataset.
Get data-quality report	GET /api/v1/datasets/{dataset_id}/quality-report	Retrieve validation and quality metrics for a dataset.
Dashboard summary	GET /api/v1/dashboard/summary	Return selected KPI/summary values for the requested filters.
Trend analysis	GET /api/v1/analytics/trends	Return time-series values for a selected indicator and optional dimensions.
Chatbot question	POST /api/v1/chat/query	Submit a natural-language question and return a data-grounded answer, if chatbot capability is implemented.

8.4.1 Proposed endpoint contracts and examples

1. GET /api/v1/health — no request body. Proposed response: {"status":"ok","service":"ewii-api"}. If database readiness is checked, include a dependency status such as {"database":"available"}; this response must reflect a real connectivity check rather than a hard-coded value.
2. POST /api/v1/datasets — multipart form upload with a file field and optional metadata fields such as source_name and domain. Proposed success response (202 Accepted): {"dataset_id":"ds_001","status":"received","message":"Dataset queued for validation"}. The identifier and status are illustrative.
3. POST /api/v1/datasets/{dataset_id}/validate — no body required for a previously registered dataset. Proposed response: {"dataset_id":"ds_001","valid":true,"checks":{"required_columns":true,"data_types":true,"missing_values":false},"issues":["Missing values found in column value"]}. Example values are illustrative; a real response must be calculated from the uploaded data.
4. POST /api/v1/datasets/{dataset_id}/process — proposed JSON body: {"mode":"clean_transform_load"}. Proposed response (202 Accepted): {"dataset_id":"ds_001","status":"processing"}. A background-job mechanism is a future design choice and must be selected before asynchronous behavior is promised.
5. GET /api/v1/datasets/{dataset_id}/quality-report — proposed response: {"dataset_id":"ds_001","row_count":1200,"duplicate_rows":3,"missing_value_counts":{"value":8},"validation_status":"review_required"}. Counts are example placeholders, not project measurements.
6. GET /api/v1/dashboard/summary?year=2024&area=Egypt — proposed response: {"filters":{"year":2024,"area":"Egypt"},"kpis":[{"name":"total_indicator_value","value":123.4,"unit":"example"}]}. The KPI names and calculation definitions must be agreed from the project data dictionary.
7. GET /api/v1/analytics/trends?indicator=water_use&from_year=2010&to_year=2024 — proposed response: {"indicator":"water_use","unit":"example_unit","series":[{"year":2010,"value":10.2},{"year":2024,"value":12.8}]}. The example series is illustrative only.

8. POST /api/v1/chat/query — proposed JSON body: {"question":"How has the selected water indicator changed over time?"}. Proposed response: {"answer":"The indicator increased over the selected period.","data_references":[{"table":"Fact_Water_Indicators","filters":{"year":"2010–2024"}}]}. The answer and reference structure depend on the eventual retrieval and response-generation design.
Common proposed error envelope:
{"detail":{"code":"VALIDATION_ERROR","message":"The request could not be processed.","issues":["A required field is missing."]}}. Suggested status codes include 200 for successful reads, 202 for accepted processing, 400 for invalid input, 404 for unknown dataset/indicator, 422 for schema validation errors, and 500 for unexpected server errors. These are design recommendations, not implemented behavior.
Before implementation, define the permitted file types and size limits, authentication/authorization policy, pagination, filtering vocabulary, KPI definitions, dataset lifecycle states, and whether long-running processing is synchronous or queued. FastAPI's interactive API documentation can be enabled during development once routes and schemas exist; it is not currently available.

8.5 Testing and Validation Plan

All tests in this section are proposed. No unit, integration, end-to-end, performance, security, or user acceptance tests are reported as executed. Test evidence should be recorded only after the relevant components have been implemented and the test has actually been run.

8.5.1 Unit testing plan

Unit tests should isolate individual functions and service modules using controlled inputs and expected outputs. Candidate test areas include file metadata checks, required-column and data-type validation, missing-value and duplicate detection, transformation rules, KPI calculations, query-parameter validation, response-schema serialization, and exception handling. Tests should cover valid, invalid, empty, boundary, and unexpected inputs. The team may select a Python testing framework during implementation; no framework is currently confirmed.

Test ID	Module / scenario	Test input or action	Expected result (proposed)
UT-01	Required-column validation	Dataset contains all required columns.	Validation reports required columns present.
UT-02	Missing required column	Remove a mandatory column from a sample dataset.	Validation fails with the missing column identified.
UT-03	Data-type validation	Provide text where a numeric field is required.	Validation reports a type issue without silently coercing invalid data.
UT-04	Missing values and duplicates	Use a controlled dataset with known nulls and duplicate rows.	Reported counts match the known test fixture.
UT-05	Transformation logic	Apply documented cleaning/transformation rules to a small fixture.	Output values and schema match the expected fixture.
UT-06	KPI calculation	Supply a small dataset with manually calculated expected aggregates.	Returned KPI values match the independently calculated results.
UT-07	Invalid API parameters	Pass an unsupported year range or unknown indicator.	A clear client error is returned; no unhandled exception occurs.
UT-08	Database error handling	Simulate a failed repository operation.	Service returns a controlled error and does not expose credentials or SQL details.

8.5.2 Integration testing plan

Test ID	Integration path	Test steps	Expected result (proposed)
IT-01	Streamlit ↔ FastAPI	Run both services in a test environment; submit a dashboard filter from the UI.	Streamlit sends the expected request and renders the returned data or a controlled error.
IT-02	FastAPI ↔ SQL Server	Configure a test database and invoke a read-only health or query operation.	The API obtains a connection and executes the parameterized query; unavailable database state is reported clearly.
IT-03	Data retrieval and processing	Load a controlled sample dataset through the planned processing flow.	Validation, transformation, and database load preserve the agreed schema and record counts.
IT-04	End-to-end dashboard flow	Select filters in Streamlit, request a KPI, retrieve SQL data through FastAPI, and render the result.	The displayed KPI corresponds to the database result and selected filters.
IT-05	Data-quality report flow	Submit a dataset with known quality issues and request its report.	The report reflects the expected validation findings and processing status.
IT-06	Failure-path integration	Stop or misconfigure the API/database and repeat a UI request.	The user receives a safe, actionable error; secrets and internal stack traces are not displayed.

8.5.3 User acceptance testing (UAT)

UAT should be performed by representative Admin, Data Analyst, and End User roles against an agreed acceptance dataset and approved KPI definitions. The following scenarios are proposed; pass/fail status must be completed by testers after execution.

UAT ID / role	Scenario and steps	Expected acceptance result	Execution status
UAT-01 Admin	Upload a supported dataset, review its metadata, and submit it for validation.	The system accepts the supported file, assigns a dataset reference, and displays its processing state.	Not executed — planned
UAT-02 Admin	Submit a dataset with a missing required column or invalid type.	The system identifies the issue and prevents invalid data from being published.	Not executed — planned
UAT-03 Admin / Analyst	Open the data-quality report for a processed test dataset.	The report presents understandable quality metrics and identified issues.	Not executed — planned
UAT-04 Analyst	Open the dashboard and filter by available dimensions such as year or area.	Displayed KPIs and visualizations update consistently with the selected filters.	Not executed — planned
UAT-05 Analyst	Select an indicator and review its historical trend.	The chart and values match the approved query/result for the selected period.	Not executed — planned
UAT-06 End User	Submit a natural-language question through the chatbot (if included in the release).	The response is relevant to retrieved project data and exposes appropriate context or references.	Not executed — conditional

UAT-07 Admin / Analyst	Trigger a request while the API or database is unavailable in the test environment.	The interface communicates the failure clearly and does not show sensitive technical details.	Not executed — planned

Acceptance governance: define a test owner, environment, build/version, test data, execution date, observed result, defect reference, and approval for each case. A release should not be marked accepted until critical defects are resolved or formally waived.

8.6 Deployment Strategy

The project is currently at the planning/design stage for the three selected application technologies. No hosting environment, deployment pipeline, environment variables, secrets store, production database, or live service has been configured. The following is a proposed deployment approach.

8.6.1 Hosting environment

Component	Suitable hosting options (proposed)	Key considerations
Streamlit frontend	A managed Streamlit-compatible hosting service, or a container/VM in a cloud or institutional environment.	HTTPS termination, outbound access to the API, session capacity, secrets configuration, and controlled access.
FastAPI backend	A managed container application service, Kubernetes only if operational scale justifies it, or a Linux/Windows VM running an ASGI server behind a reverse proxy.	Private connectivity to SQL Server, health checks, logs, process management, and horizontal scaling where supported.
Microsoft SQL Server	Managed SQL Server-compatible database service, or SQL Server on a secured institutional/cloud VM.	Private networking, backups, restore testing, encryption, patching, database sizing, and licensing.
Development / test	Separate local or shared development environment and an isolated test database.	Use synthetic or masked data; do not share production credentials with development.

The final hosting choice should account for institutional policies, available budget, SQL Server licensing, data residency, expected concurrent users, and whether the project must be accessible outside the institution's network.

8.6.2 Proposed deployment process

• Create and review the SQL Server schema from the approved logical and physical models; apply version-controlled SQL migration scripts to a development database first.
• Create the Python backend project, implement the agreed service modules and API contracts, and pin dependencies in a project dependency file.
• Configure the FastAPI service to read database connection settings from environment variables or a managed secrets service; verify connectivity using a non-production database.
• Build the Streamlit interface and configure its API base URL and timeout behavior through environment-specific settings.
• Run automated unit and integration tests against isolated test data and a test database; resolve critical defects before promotion.
• Deploy the API and frontend to their selected hosting environments; expose only the required HTTPS endpoints and keep SQL Server on a private network.
• Apply production database migrations through a controlled release process, configure backups and monitoring, and perform smoke tests after deployment.
• Document rollback steps for application releases and database changes; retain the previous known-good application version.

8.6.3 Proposed CI/CD pipeline

No CI/CD pipeline currently exists. A future pipeline may be implemented using the team's selected Git hosting and automation platform. A recommended sequence is:

Stage	Proposed automated activity	Release gate
1. Commit / pull request	Code formatting and static checks; validate dependency lock/configuration files.	Review and approval required before merge.
2. Unit tests	Run backend service, validation, transformation, and calculation tests.	All required unit tests pass.
3. Build	Build versioned backend and frontend artifacts or container images.	Build completes reproducibly.
4. Integration tests	Deploy to a temporary test environment and test API/database/UI communication.	Critical integration tests pass.
5. Security checks	Scan dependencies and container images; check for accidental secrets in commits.	No unresolved critical/high findings under the team's policy.
6. Deploy to staging	Apply test migrations and deploy candidate artifacts.	Smoke tests and stakeholder review pass.
7. Production approval	Require authorized manual approval and a release record.	Approved change window and rollback plan.
8. Deploy and verify	Deploy application, apply approved migrations, run health/smoke checks, monitor logs.	Rollback if health checks or critical user journeys fail.

8.6.4 Configuration and secrets management

• Use separate configuration values for development, testing, staging, and production; do not commit credentials or private connection strings to source control.
• Store SQL Server host, database name, username or managed identity settings, encryption requirements, and API configuration in environment variables or a managed secrets vault.
• Prefer a least-privilege database identity with only the permissions required by the application; separate read and write permissions where practical.
• Rotate credentials, restrict access to secret-management systems, and redact credentials, tokens, personal data, and sensitive query values from logs.
• Validate required configuration at application startup and fail safely with a clear operational message when mandatory settings are missing.

8.6.5 Scalability and performance considerations

• Scale Streamlit and FastAPI independently where the selected hosting platform supports multiple instances; keep application instances stateless where practical.
• Use bounded database connection pooling and parameterized, indexed queries; review query plans and the dimensional model as data volume grows.
• Paginate large result sets, limit upload sizes, and avoid returning full fact-table extracts to the browser when aggregated results are sufficient.
• Move long-running ingestion, cleaning, or model operations to a background worker/queue if measured execution times make synchronous API requests unsuitable.
• Cache safe, repeatable dashboard aggregates with explicit invalidation rules; do not cache user-specific or sensitive data without an access-aware design.
• Monitor request latency, error rates, database CPU/storage, connection counts, and job duration; use measurements to guide scaling rather than assuming a microservices architecture is required.

8.6.6 Security considerations

• Serve the frontend and API over HTTPS; restrict cross-origin access to the known frontend origin and expose only required API routes.
• Define role-based authorization for Admin, Data Analyst, and End User actions before implementing administrative upload or processing operations.
• Validate file type, file size, schema, and content; treat uploaded files and user-supplied questions as untrusted input.

• Use parameterized SQL and least-privilege database accounts; never construct SQL statements by concatenating user input.
• Keep SQL Server private and restrict inbound network access to the backend service or approved application subnet.
• Protect secrets and sensitive data in transit, at rest, backups, and logs; establish retention and restore procedures.
• Apply dependency updates and security reviews, and record audit events for privileged dataset and administrative operations.

8.7 Implementation Readiness and Status

Area	Current status based on project information	Next implementation milestone
Technology selection	Planned: Streamlit, FastAPI (Python), Microsoft SQL Server.	Confirm versions, hosting constraints, and dependency choices.
Backend and API	Not implemented; no live endpoints are available.	Create FastAPI project, service boundaries, schemas, and agreed route contracts.
Frontend	Not implemented; no Streamlit application is available.	Create dashboard navigation, filter components, and API client.
Database connectivity and SQL scripts	Not implemented; database connection and executable SQL scripts are not available.	Approve schema, create migrations/scripts, and test connectivity.
Testing	No tests are reported as executed.	Create test fixtures and automate unit, integration, and UAT evidence capture.
Deployment and CI/CD	No deployment or pipeline is configured.	Select hosting, configure secure environments, and establish a staged release workflow.

This implementation plan should be updated as design decisions are finalized and work is completed. Future revisions should change an item from “planned” to “implemented” only when supported by repository evidence, deployed configuration, or recorded test results. The proposed endpoint names and examples in Section 7.4 must not be presented as existing API functionality.